

Decision Trees : pre-reads

1. Decision Tree Intuition

Description:

- A decision tree is a **tree-like model** where each internal node represents a feature-based decision, each branch represents a decision outcome, and each leaf represents a prediction.
- It splits data recursively to separate classes or predict numerical values.

Importance:

- Highly interpretable: you can follow the path from root to leaf to understand predictions.
 - Useful for problems where logic and rules are important, e.g., credit scoring.
-

2. How to Split the Nodes

Description:

- Nodes are split to **maximize information gain** or minimize impurity (Gini or Entropy).
- Decision depends on which feature best separates the classes at that point.

Importance:

- Correct splits create pure nodes, leading to better accuracy.
 - Understanding splits is key to controlling overfitting.
-

3. Entropy

Description:

- Entropy measures **disorder or uncertainty** in a dataset.

- Formula (conceptual): High entropy → classes are mixed, Low entropy → classes are pure.

Importance:

- Used in **Information Gain** to decide the best feature for splitting.
 - Helps visualize why trees split on certain features.
-

4. Building a Decision Tree Intuition

Description:

- Start with the root node containing all data.
- Choose the best feature using entropy or Gini to split.
- Repeat recursively until stopping criteria (max depth, minimum samples) are reached.

Importance:

- Helps understand **recursive partitioning**.
 - Guides expectations on tree depth, overfitting, and interpretability.
-

5. Visualizing the Process of Building DT

Description:

- Trees can be drawn to show **splits at each node**, decision paths, and leaf predictions.
- **Tools:** matplotlib, Graphviz, `plot_tree` in sklearn.

Importance:

- Makes the decision process **transparent and explainable**.
- Useful for debugging and feature analysis.

Part 3: Decision Tree Implementation

6. Sklearn Implementation

Description:

- Use `DecisionTreeClassifier` for classification, `DecisionTreeRegressor` for regression.
- Key parameters: `max_depth`, `min_samples_split`, `criterion` (entropy/gini), `max_features`.

Importance:

- Simplifies experimentation and deployment.
- Allows hyperparameter tuning to optimize performance.

7. Gini Impurity

Description:

- Gini measures **how often a randomly chosen element would be incorrectly classified**.
- Lower Gini → purer node; higher Gini → more mixed.

Importance:

- Alternative to entropy for splitting decisions.
- Computationally simpler than entropy in practice.

8. Overfit vs. Underfit

Description:

- **Overfit:** Tree fits training data perfectly but fails on unseen data (too deep).
- **Underfit:** Tree is too shallow, fails to capture patterns (too simple).

Importance:

- Understanding overfit/underfit guides hyperparameter tuning: depth, min samples, pruning.
 - Critical for robust, generalizable models.
-

9. Hyperparameter Tuning

Description:

- Adjust parameters like `max_depth`, `min_samples_leaf`, `min_samples_split`, `max_features`.
- Use cross-validation to find optimal values.

Importance:

- Prevents overfitting, improves model generalization.
 - Balances interpretability vs accuracy.
-

10. Visualizing Decision Trees

Description:

- Use `plot_tree` or export to Graphviz.
- Can show feature splits, thresholds, impurity at each node, sample counts, and predicted class.

Importance:

- Helps explain predictions to non-technical stakeholders.
 - Useful in feature selection and model auditing.
-

11. Feature Importance

Description:

- Decision Trees calculate **feature importance** based on how much a feature reduces impurity across all splits.
- Can be accessed via `feature_importances_` in sklearn.

Importance:

- Identifies **key drivers** in the data.
- Useful for feature selection, business insights, and model simplification.